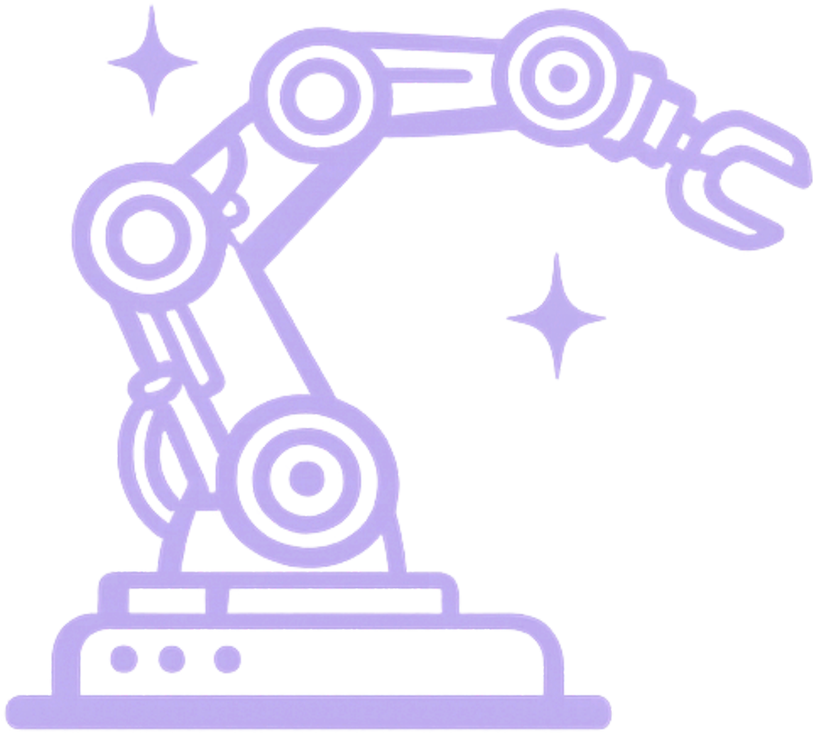


PatchWork

Diya Bengani, Alexandra Yankovskaya, Vera Chuang,
Elias Assalif

19 April 2026

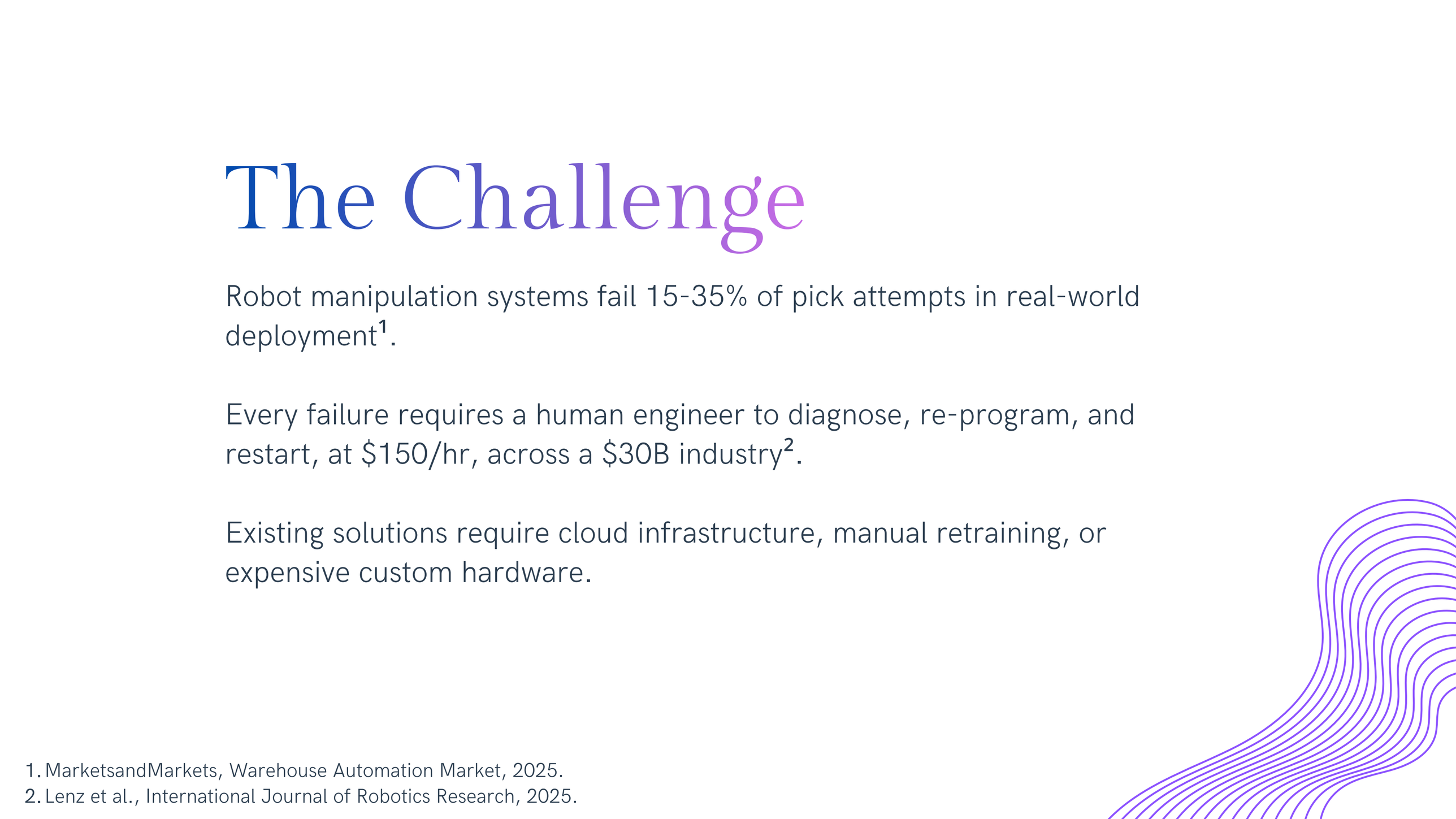
StarkHacks @ Purdue University

The Challenge

Robot manipulation systems fail 15-35% of pick attempts in real-world deployment[1].

Every failure requires a human engineer to diagnose, re-program, and restart, at $150/hr, across a $30B industry[2].

Existing solutions require cloud infrastructure, manual retraining, or expensive custom hardware.



1. MarketsandMarkets, Warehouse Automation Market, 2025.
2. Lenz et al., International Journal of Robotics Research, 2025.

PatchWork

- An on-device AI execution layer for robot arms.
- Compiles natural language commands into structured skill programs
- Executes and verifies each action in real time using a vision-language model running entirely on AMD silicon.

When a step fails, PatchWork diagnoses the root cause, patches the program automatically, and retries: **no human intervention, no cloud dependency, no retraining.**

Market Opportunity

Market Size

The robotics maintenance market is estimated to be worth **$34 billion**, presenting an opportunity for significant revenue generation and growth within the industry.

Intellectual Property

This project has strong intellectual property potential with proprietary technology that can create a competitive advantage in the evolving robotics landscape.

Industry Demand

There is a growing demand for advanced robotic solutions in manufacturing and logistics automation, driven by the need for efficiency and reduced downtime.

How PatchWork Works

1 Command

Operator gives a natural-language instruction.

2 Plan

Phi-3 converts it into structured skill steps.

3 Execute

LeRobot replays the recorded robot motion.

4 Verify

Moondream checks whether each step succeeded.

5 Diagnose

Classifier identifies the likely failure type.

6 Patch

Apply corrective changes from patch memory.

7 Retry / Escalate

Retry automatically up to 2 times, then flag review.

Runs locally on AMD silicon

- Phi-3 for planning
- Moondream for verification
- No cloud dependency
- Persistent patch memory

plan -> act -> verify -> patch -> retry

Tech Stack

Hardware

- AMD Ryzen AI 9 HX 370
- AMD Radeon 890M
- AMD XDNA 2 NPU
- ROCm 6.3 (AMD)
- VitisAI Execution Provider (AMD)
- LeRobot SO-101
- USB webcam

AI Models

- Moondream2: 1.7 B param model, running on AMD Radeon 890M via Ollama
- Phi-3-mini: NLP to robot control JSONs

Software Architechture

- **Closed-loop execution engine**: step-by-step verification after every robot motion
- **Failure classifier:** rule-based root cause detection (grasp error, placement miss)
- **Patch library**: persistent execution memory, fixes stored and pre-applied on future runs
- **Streamlit dashboard**: live GPU latency, step visualization, patch memory

Infrastructure

- LeRobot 0.4.1
- Python 3.10
- Ubuntu 24.04

AMD

GPU Optimization

Real-time VLM needs sub-second inference.
Moondream2 run locally via Ollama.
Initial benchmark on CPU:

```
2395ms | tokens:40 | The image features a gray square with a white border, which
```

Ollama: Offloaded to AMD Radeon 890M
→ No cloud dependency, runs entirely on AMD Silicon

```
load_tensors: offloaded 25/25 layers to GPU
load_tensors:        ROCm0 model buffer size =   732.30 MiB
clip_ctx: CLIP using ROCm0 backend
```

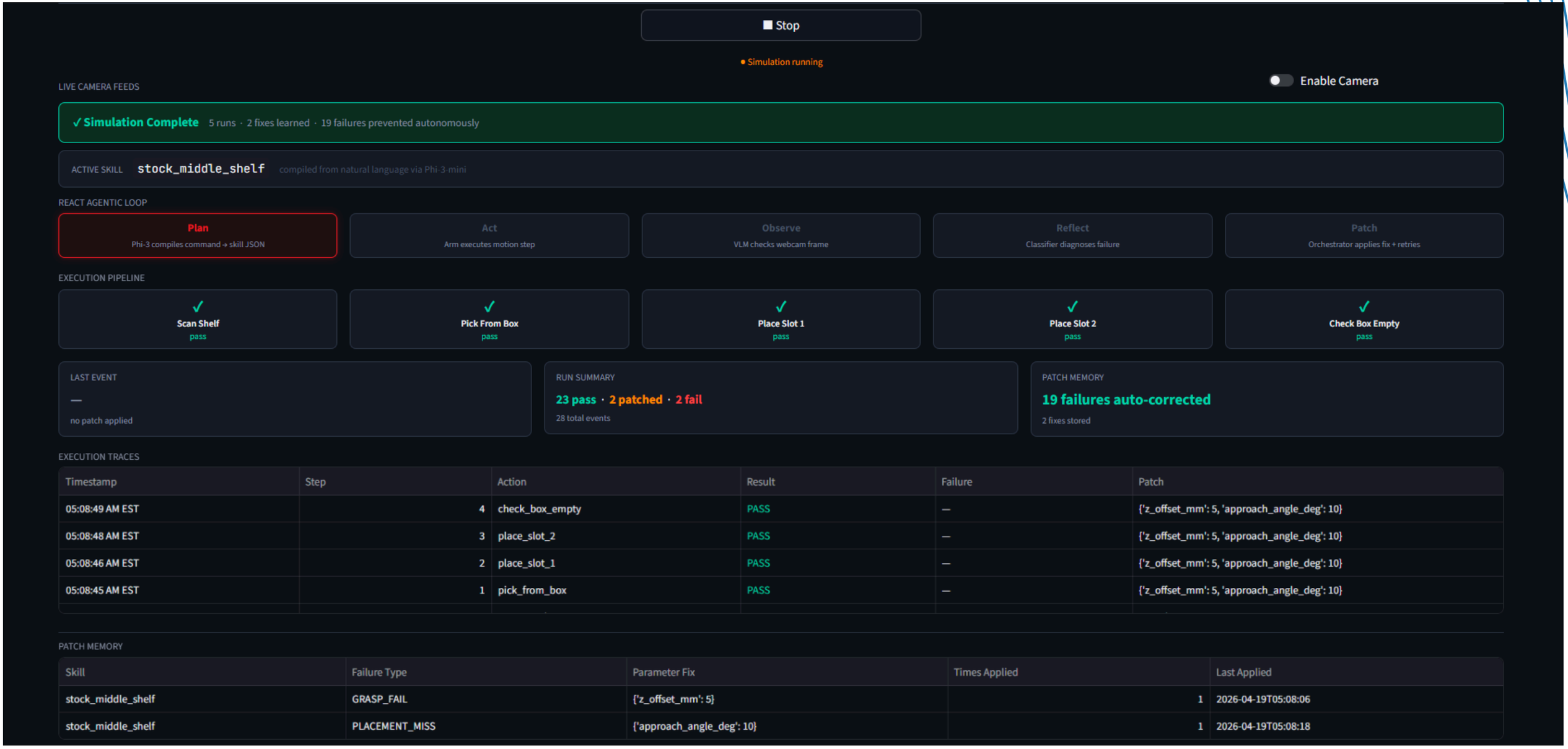
Custom ModelFile template (real world, no caching):

```
554ms total | 453ms image encoding | 26ms generation | 3 tokens
```

Grammar-constrained decoding using JSON schema:

```
827ms cold | 199ms warm | {"answer": "no", "confidence": "high"}
```

DashBoard


Stop
Simulation running
LIVE CAMERA FEEDS
Enable Camera
✓ Simulation Complete 5 runs · 2 fixes learned · 19 failures prevented autonomously
ACTIVE SKILL stock_middle_shelf compiled from natural language via Phi-3-mini
REACT AGENTIC LOOP
Plan
Phi-3 compiles command → skill JSON
Act
Arm executes motion step
Observe
VLM checks webcam frame
Reflect
Classifier diagnoses failure
Patch
Orchestrator applies fix + retries
EXECUTION PIPELINE
✓ Scan Shelf pass
✓ Pick From Box pass
✓ Place Slot 1 pass
✓ Place Slot 2 pass
✓ Check Box Empty pass
LAST EVENT
—
no patch applied
RUN SUMMARY
23 pass · 2 patched · 2 fail
28 total events
PATCH MEMORY
19 failures auto-corrected
2 fixes stored
EXECUTION TRACES
Timestamp Step Action Result Failure Patch
05:08:49 AM EST 4 check_box_empty PASS — {'z_offset_mm': 5, 'approach_angle_deg': 10}
05:08:48 AM EST 3 place_slot_2 PASS — {'z_offset_mm': 5, 'approach_angle_deg': 10}
05:08:46 AM EST 2 place_slot_1 PASS — {'z_offset_mm': 5, 'approach_angle_deg': 10}
05:08:45 AM EST 1 pick_from_box PASS — {'z_offset_mm': 5, 'approach_angle_deg': 10}
PATCH MEMORY
Skill Failure Type Parameter Fix Times Applied Last Applied
stock_middle_shelf GRASP_FAIL {'z_offset_mm': 5} 1 2026-04-19T05:08:06
stock_middle_shelf PLACEMENT_MISS {'approach_angle_deg': 10} 1 2026-04-19T05:08:18

Next Steps

NPUs!

2h away from having a working version of NPU running Phi-3-mini compiler!

NPU-accelerated failure embedding and patch lookup.
Embedding model TBD

Human in the Loop

Currently: no failsafe, model keeps trying if it fails.

Industrial environments:
Confidence-gated verification

Low confidence: HIL
High confidence: autonomy

Parallel inference using CPU + GPU + NPU

Currently: GPU +CPU sequentially

Future: NPU runs Phi-3-mini compiler, GPU runs Moondream2 verifier, CPU runs the orchestrator state machine.